

Mobile Development :

11 : Integration of External Services : Part 2

Working with a Case Study



Professor Imed Bouchrika

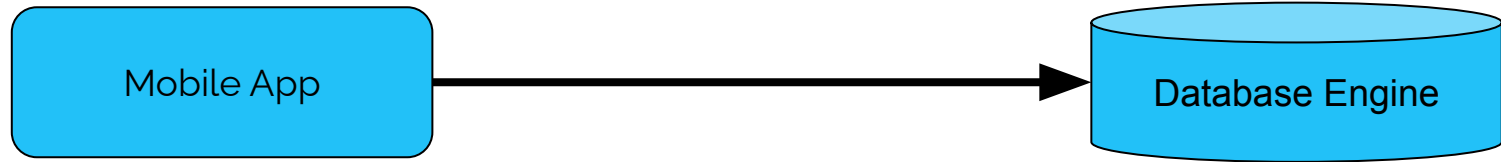
National School of Artificial Intelligence
imed.bouchrika@ensia.edu.dz

Outline :

- **Recap from Previous Lecture**
 - *Supabase Access*
 - *Backends*
 - *Background Jobs*
- **External Technologies**
 - *Firebase Messaging*
 - *Firebase Remote Config*
 - *Firebase Functions ?*
- **Accessing Mobile Hardware**
 - *Permissions*
 - *Accessing Hardware*

External Databases & Storage : Supabase

- How to access a database from a mobile app ?
 - Direct Database Access



```

Future<bool> init my app() async {
  final sl = GetIt.instance;
  if (1 == 1) {
    await Supabase.initialize(
      url: 'https://tstyetrnwndanxhqjafk.supabase.co',
      anonKey: 'sb_publishable_Z82M6XDWOxZ2lFdtFapd9Q_TIIP2wab',
    );
    sl.registerLazySingleton<TodoRepo>(() => TodoSupabaseRepo());
  } else {
    sl.registerLazySingleton<TodoRepo>(() => TodoFirestoreRepo());
    await Firebase.initializeApp(
      options: DefaultFirebaseOptions.currentPlatform,
    );
  }
  WidgetsFlutterBinding.ensureInitialized();
  return true;
}

void main() async {
  await init my app();
  runApp(const MainApp());
}

class MainApp extends StatelessWidget {
  const MainApp({super.key});
  @override
  Widget build(BuildContext context) {
    return MultiBlocProvider(
      providers: [BlocProvider(create: (context) => TodoCubit())],
      child: MaterialApp(home: HomeScreen()),
    );
  }
}

```

lib	1
cubits	1
todo_cubit.dart	1
repo	
todo_firestore_repo.dart	
todo_repo.dart	
todo_supabase_repo.dart	
screens	
bottom_form.dart	
homescreen.dart	2
todo_card.dart	1
firebase_options.dart	
main.dart	1
> linux	

```
class TodoSupabaseRepo extends TodoRepo {
  Future<List<Map<String, dynamic>>> getData() async {
    try {
      final supabase = Supabase.instance.client;
      List<Map<String, dynamic>> response = await supabase
        .from('todos')
        .select()
        .order('created_at', ascending: false);
      print('_____ $response');
      return response;
    } catch (e, stack) {
      print('Error fetching data from Supabase: $e $stack');
      return [];
    }
  }

  Future<bool> deleteRecord(String id) async {
    return true;
  }

  Future<bool> insertRecord(String title) async {
    final supabase = Supabase.instance.client;
    await supabase.from('todos').insert({'title': title});
    return true;
  }
}
```

lib	
cubits	
todo_cubit.dart	1
repo	
todo_firestore_repo.dart	
todo_repo.dart	
todo_supabase_repo.dart	
screens	
bottom_form.dart	
homescreen.dart	2
todo_card.dart	1
firebase_options.dart	
main.dart	1
> linux	

External Databases &

Personally, I always prefer access to the DB to be done via an EndPoint Server where extra business logic can be performed. This involves:

- Curation of Data,
- Validation
- Advanced Business Logic (Machine Learning..)
- Communication with Other services.
- Better control of security
- Better extensibility and easy to roll new features
- Easy to change the providers, technologies
 - ? Just the complexity you will be overloading the Endpoint, deploy more instances to overcome the problem

Development time	Execution	Take time

External Databases &

Direct Database Access from your mobile app :

- For simple and disposable apps having no intention to maintain them in the future.
- What happens if you are forced to change the database provider ? or just a database table **even a column name** inside the database table ?
- Security ? it is too complex to enforce and master. What happens if we can declassse your app and find your keys ? we will flood your database with requests and you end up paying a large bill.

Development Time	Excellent	Takes time
------------------	-----------	------------

```

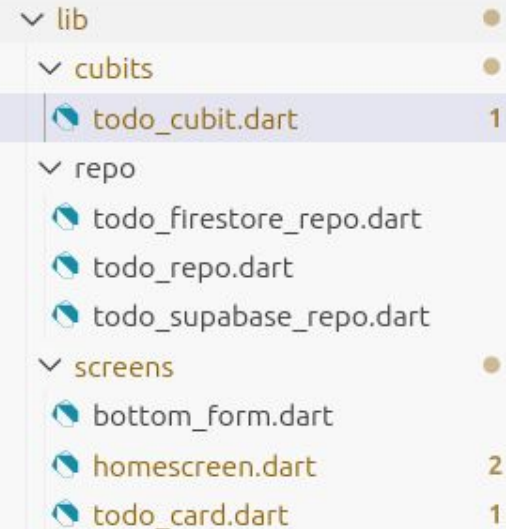
class TodoSupabaseRepo extends TodoRepo {
  Future<List<Map<String, dynamic>>> getData() async {
    try {
      final supabase = Supabase.instance.client;
      List<Map<String, dynamic>> response = await supabase
        .from('todos')
        .select()
        .order('created_at', ascending: false);
      print('_____ $response');
      return response;
    } catch (e, stack) {
      print('Error fetching data from Supabase: $e $stack');
      return [];
    }
  }

  Future<bool> deleteRecord(String id) async {
    return true;
  }

  Future<bool> insertRecord(String title) async {
    final supabase = Supabase.instance.client;
    await supabase.from('todos').insert({'title': title});
    return true;
  }
}

```

todo_supabase_repo.dart



**Will not be accepted in
the mobile dev project
final version**



Backends

Backends for a Mobile App

A decorative graphic in the top right corner consisting of a pink bar, a dark grey bar, a blue bar, and a stylized blue arrow pointing downwards and to the right, with a yellow bar partially visible on the far right.

- **What's a Backends :**

- The component where mobile apps connects to do:
 - Store/Get data
 - Delegate business logic to be conducted
- Simply,
 - It is like the kitchen where food gets cooked.
- The User sees the menu (mobile app) and should never be given access to the kitchen where all the business logic is conducted.

Backends for a Mobile App



- **Types of Backends :**

- **Backend-as-a-Service (Baas)**

- Pre-built backend services that handle common backend needs without writing server code. Think of it as "backend in a box."

- Examples:

- Firebase
 - Supase

Backends for a Mobile App



- **Types of Backends :**

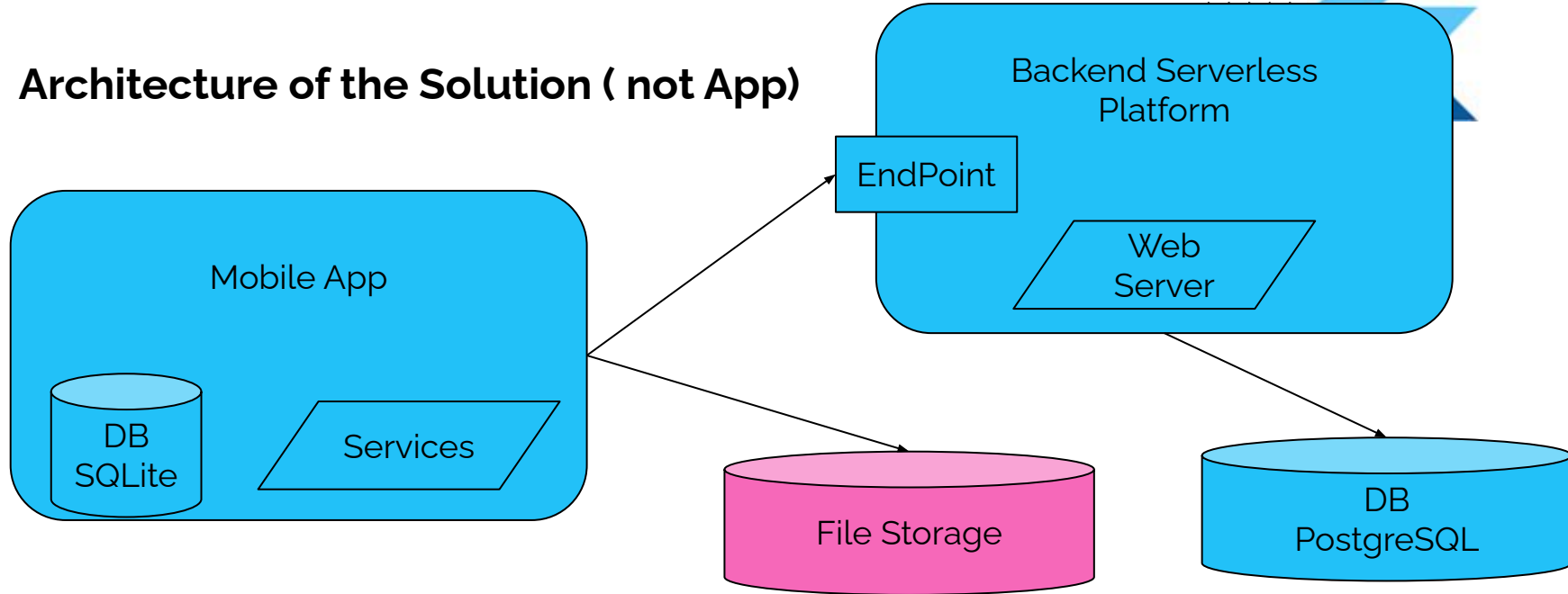
- **Custom Backend**

- The backend business logic is coded either from scratch or on top of an existing framework including:

- *Flask / Django*
 - *Node.js* + *Express*
 - *Laravel*
 - *Spring Boot*
 - *Go*

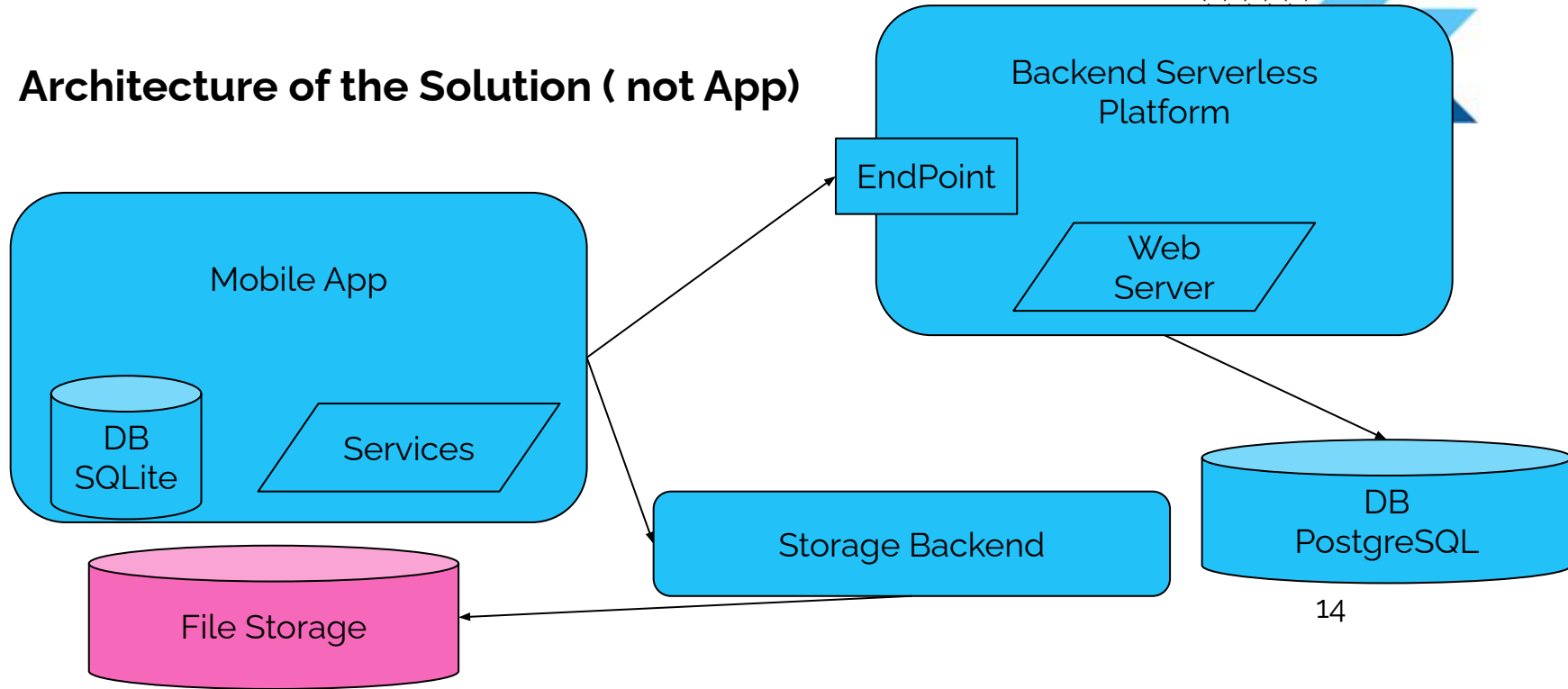
Backends for a Mobile App

- Architecture of the Solution (not App)



Serverless Technology for Mobile App Backends

- Architecture of the Solution (not App)



Backends for a Mobile App



- **How to run it : Server-Centric approach**

- Get a dedicated Server or even virtual ...
- You need to manage it yourself
 - Upgrades
 - Security
- Scaling, can be difficult to main
- But, you have control over everything.
- Cost : Cannot say (it may cost \$5/month ..)

Backends for a Mobile App

A decorative graphic in the top right corner consisting of a pink bar, a dark grey bar, a blue bar, and a stylized blue arrow pointing downwards and to the right. There are also some dotted lines and a yellow bar on the right side.

- **Introduction to Serverless Technologies**

- Even though, it is called serverless, the server is always there but:
 - Provided when there is a request from a user
 - You pay usually per request / processing duration / bandwidth transferred.
- Serverless Technologies provided by the hosting provider where they have the infrastructure to deploy dynamically servers/frameworks on demand.

Backends for a Mobile App



- **Introduction to Serverless Technologies**

- **Benefits..**

- Maintenance of the technology taken care by the infrastructure provider
 - Scaling is always being taken care of. When more users, the hosting provider will deploy more resources dynamically, (but be prepared to pay more)

- **Drawbacks..**

- You have no control
 - Dependent on a technology provider

Serverless Technology for Mobile App Backends



- **Improvement to the previous implementation:**
 - Use Python,PHP, Java,,,,for Backend instead :
 - Existing Framework : **Flask** (Or even Django)
 - Link with PostgreSQL provided by Supabase
 - Use Better Storage Facilities : S3 Storage or Firebase Storage
 - Integrate Firebase Services
 - Users' Authentication.

Serverless Technology for Mobile App Backends

- Hello World in Flask

```
from flask import Flask
import json

app = Flask(__name__)

@app.route('/categories.get')
def get_categories():
    data=[{'id':1,'name':'Food'},{'id':2,'name':'Beverage'}]
    return json.dumps(data)

@app.route('/')
def index():
    return 'Welcome ENSIA Students from Flask!'

if __name__ == "__main__":
    app.run(port=8080)
```

This is the route which **binds URL** to a function to perform a given business logic

`__name__` is a special variable when the script is invoked directly (not imported), the `__name__` is set as `"__main__"`

Serverless Technology for Mobile App Backends

- Getting Variables from URLs

```
from flask import Flask
import json

app = Flask(__name__)

@app.route('/categories.getById/<int:cat_id>')
def get_category_byId(cat_id):
    // some business logic here...
    return json.dumps(data)

@app.route('/')
def index():
    return 'Welcome ENSIA Students from Flask!'

if __name__ == "__main__":
    app.run(port=8080)
```

You can enclose URL variables inside
<type:variable_name> or simply
<variable_name>

Serverless Technology for Mobile App Backends

- Getting GET/POST Variables

```
from flask import Flask , request
import json

app = Flask(__name__)

@app.route('/user.login', methods=['GET', 'POST'] )
def users_login():
    username=request.form.get('username')
    password=request.form.get('password')
    //some more business logic
    return json.dumps(data)

@app.route('/')
def index():
    return 'Welcome ENSIA Students from Flask!'

if __name__ == "__main__":
    app.run(port=8080)
```

GET or POST variables can be accessed
via the special variable :

request.form.get(VAR_NAME)

Serverless Technology for Mobile App Backends

- Getting Binary Files from Uploads

```
from flask import Flask
import json

app = Flask(__name__)

@app.route('/photo.upload', methods=['POST'])
def users_login():
    if request.method=='POST':
        file=request.files.get('file_name')

        //some more business logic
        return json.dumps(data)

@app.route('/')
def index():
    return 'Welcome ENSIA Students from Flask!'

if __name__ == "__main__":
    app.run(port=8080)
```

Binary Data Files can be accessed via the special variable :

request.files.get(FILE_NAME)

Serverless Technology for Mobile App Backends

- Linking with Flask

```
from flask import Flask, request
import json
from supabase import create_client, Client

app = Flask(__name__)

url="https://yvntxteoqxvamhbpvd.supabase.co"
key="eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6InI2bmN0eHRlb3F4dmFtaGJ6cHZkIiwicm9sZSI6ImFub24iLCJpYXQiOiE3MDIzMDU0ODQsImV4cCI6MjAxNzg3Nzg4NH0.4AoGpxSQF3-T4b_dJ2B5ZfJY1pukT7Gu8xbKq8pN9gA"
supabase: Client = create_client(url, key)

@app.route('/todo.get')
def api_categories_get():
    response = supabase.table('todos').select("*").execute()
    return json.dumps(response.data)
```

Test and Run always Locally
FIRST

flask --app api/index run

```

from flask import Flask, request
import json
import psycopg2

app = Flask(  name  )
#This is the very stupid way to store private/confidential data inside GIT
url="db.yvnctxteoqxvamhbpvd.supabase.co"
password="Your Initial Password here..."

@app.route('/categories.get')
def api_categories_get():
    conn=False
    try:
        conn = psycopg2.connect("dbname='postgres' user='postgres' host='"+url+"' password='"+password+"'")
    except Exception as error:
        print("I am unable to connect to the database")
        return 'cannot connect to database'+str(error)

    curs=conn.cursor()
    curs.execute("select id,title from todo")
    data=[]
    for record in curs:
        print(record)
        data.append({'id':record[0], 'name':record[1]})

    return json.dumps(data)

```




Local Background Services

Local Background Services

- **Why use Background Jobs inside a mobile app :**

- Executing Heavy/Intensive tasks
- Syncing data with server
- Checking for updates
- Clearing cache
- Sending analytics
- ...

Local Background Services

- Libraries to use :

- *workmanager*

- *Cross-platform, battery efficient*
 - ***No precise timing, 15-min minimum***
 - ***Android/iOS only.***
 - *Use for: periodic sync, background updates*

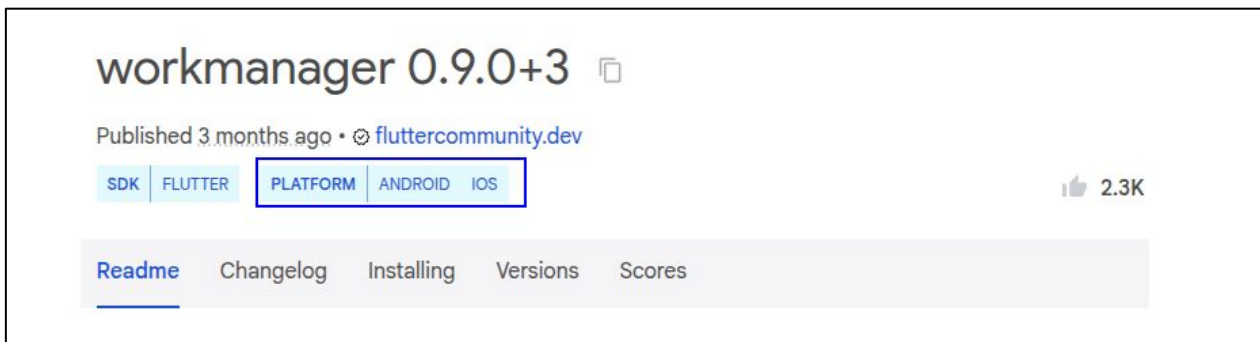


Local Background Services

- Libraries to use :

- *workmanager*

- *flutter pub add workmanager*



```

class CountScreen extends StatefulWidget {
  const CountScreen({super.key});
  @override
  State<CountScreen> createState() => _CountScreenState();
}

class CountScreenState extends State<CountScreen> {
  var increment = 0;
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('My Counter')),
      body: Center(
        child: Column(
          children: [
            Text("$increment"),
            ElevatedButton(
              onPressed: () {
                increment += 1;
                setState(() {});
                Workmanager().registerOneOffTask(
                  "check $increment", // Unique task name
                  "save_increment_db", // Task identifier
                  inputData: {
                    'message': 'saving later value',
                    'data': increment,
                  },
                );
              },
            ),
            child: Text('Increment'),
          ],
        ),
      ),
    );
  }
}

```

```

@pragma('vm:entry-point')
void callbackDispatcher() {
  Workmanager().executeTask((task, inputData) {
    print("Background task executed: $task");
    print("Input data: $inputData");

    if (task == "periodicTask") {
      print("Running period Task...");
      syncDataToServer(inputData);
    } else if (task == "save_increment_db") {
      syncDataToServer(inputData);
    }
    return Future.value(true);
  });
}

Future<bool> init my app() async {
  Workmanager().registerPeriodicTask(
    "periodicTask", // Unique task name
    "periodicTask", // Task identifier
    frequency: const Duration(minutes: 15),
    inputData: {'message': 'Some periodic task from the Counter'},
  );
  return true;
}

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await init my app();
  runApp(MainApp());
}

```

Local Background Services

- Libraries to use :

- ***android_alarm_manager_plus***

- *Precise timing, can run at 8 AM*
 - ***Android only***
 - *Use for: alarms, reminders, scheduled tasks*

Local Background Services

- Libraries to use :

- ***android_alarm_manager_plus***

- *flutter pub add android_alarm_manager_plus*




```
@pragma('vm:entry-point')
void sevenAMCallback() {
    print("Good morning! It's 7 AM!");
}
```

```
final now = DateTime.now();
DateTime scheduledTime = DateTime(now.year, now.month,
now.day, 7, 0);
await AndroidAlarmManager.periodic(
    const Duration(days: 1), // Repeat every 24 hours
    0,
    sevenAMCallback,
    startAt: scheduledTime,
    exact: true,
    wakeup: true,
    rescheduleOnReboot: true,
);
return true;
}
```

```
final now = DateTime.now();
DateTime scheduledTime = DateTime(now.year,
now.month, now.day, 7, 0);
await AndroidAlarmManager.oneShotAt(
    scheduledTime, // When to trigger
    0, // Unique alarm ID
    sevenAMCallback, // Function to call
    exact: true, // Trigger at exact time
    wakeup: true, // Wake device if sleeping
    rescheduleOnReboot: true, // Persist after reboot
);
```

Local Background Services

- Libraries to use :

- ***flutter_background_service***

- *Continuous operation*
 - ***High battery drain***
 - *Use for: music players, tracking apps*



Local Background Services

- Libraries to use :

- ***cron***

- *Easy to use, precise timing*
 - ***Only works when app is open***
 - *Use for: UI updates, real-time dashboards*

Feature	workmanager	android_alarm_manager_plus	flutter_background_service	cron
Platform Support	Android & iOS	Android Only	Android & iOS	All Platforms
Works When App Closed	Yes	Yes	Yes	No
Execution Type	Periodic (deferred)	Scheduled (precise)	Continuous	Scheduled (precise)
Minimum Interval	15 minutes	No limit	N/A	Seconds
Precise Timing	No (approximate)	Yes (exact)	Yes	Yes (when running)
Battery Impact	Low (optimized)	Medium	High	Low (app running)
Setup Complexity	Easy	Medium	Complex	Very Easy

Section 2

Messaging using Firebase.



Firestore Services : Messaging



- **Firestore Messaging :**

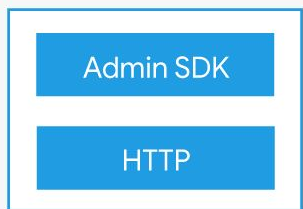
- Used either to send “almost” immediately **from remote systems:**
 - Notification Messages
 - In-App Messages (Shown inside the app whilst the user uses the app)
- You can send to :
 - A single user
 - Group of users subscribed to a specific group/**topic**.
- The sending is **usually** triggered by a **backend server** which communicates with Firestore Servers

1

Message building and
targeting



Notifications
Console GUI



Trusted
Environment

Send to topic
or instance

2

FCM
backend



3

Platform-level
message transport

Android
transport layer

iOS / APNs

Web Push

4

SDK
on device



Firebase Services : Messaging

- **Firebase Messaging : Overview for a Hello World !**

- **On Mobile App :**

- Init the Firebase Messaging Instance
- Depending on your App:
 - Get the Token for the User and send it to your Backend **OR**
 - Subscribe the User to a specific topic/channel
- Implement Event Listeners when a message arrives
 - You may even take the user to a specific screen when clicking on the message

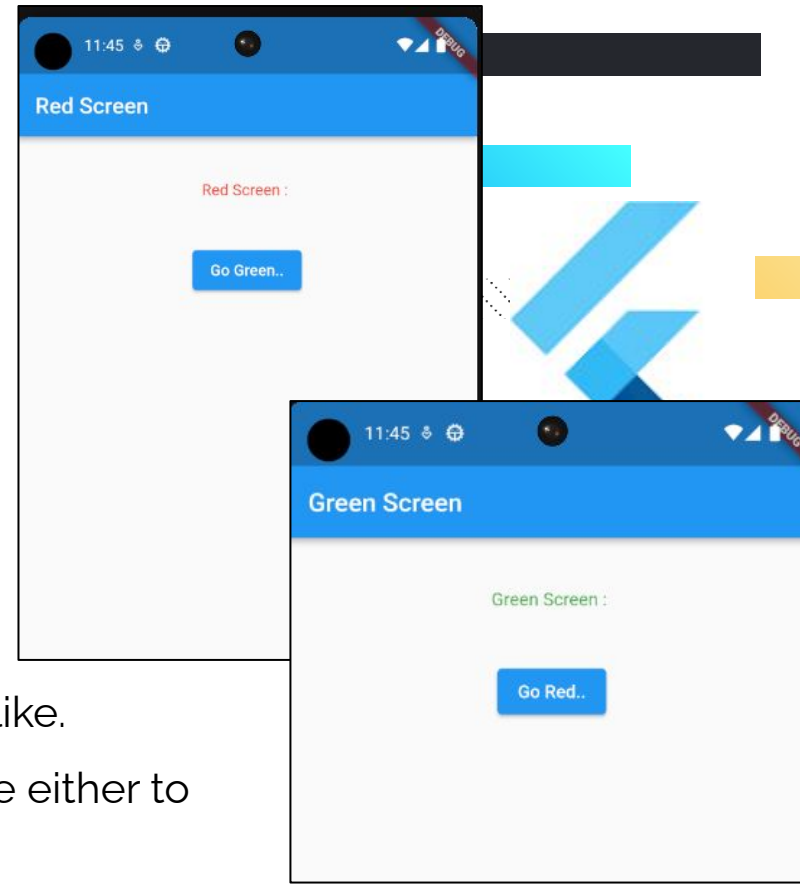
Firestore Services : Messaging

- **Firestore Messaging : Overview for a Hello World !**
 - **On Backend Server**
 - Implement UI/Service to send messages either to specific user or channel
 - You may provide an API Endpoint to get User Tokens

Firestore Services : Messaging

- **Firestore Messaging : Simple MVP**

- Apps with two Screens :
 - Red
 - Green
- Users can choose whichever screen they like.
- Messages would be sent from time to time either to
 - All users
 - Users of specific color (Red or Green)



```
import 'package:firebase_message_hello_world/screens/red.dart';
import 'package:flutter/material.dart';

void main() {
  runApp(const MainApp());
}

class MainApp extends StatelessWidget {
  const MainApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: RedScreen(message: 'Initial Loading..as RED'),
    );
  }
}
```



```
import 'package:firebase_message_hello_world/screens/green.dart';
import 'package:flutter/material.dart';
class RedScreen extends StatefulWidget {
  String message;
  RedScreen({super.key, this.message = ''});
  @override
  State<RedScreen> createState() => _RedScreenState();
}
class _RedScreenState extends State<RedScreen> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("Red Screen")),
      body: Center(
        child: Column(
          children: [
            SizedBox(height: 40,),
            Text("Red Screen : ${widget.message}",
              style: TextStyle(color: Colors.red)),
            SizedBox(height: 40,),
            ElevatedButton(
              onPressed: () {
                Navigator.of(context).pop();
                Navigator.of(context).push(
                  MaterialPageRoute(builder: (context) => GreenScreen()));
              },
              child: Text("Go Green..")
            ),
          ],
        ),
      ),
    );
  }
}
```

Firebase Services : Messaging

- **Firebase Messaging : Simple Steps for a Hello World !**

- **Mobile App :**

- Make sure that you have installed the firebase cli tools
- Within you your project terminal run:
 - **flutterfire configure**
 - Either choose an existing project or create a new one

01

```
imed@imed-Inspiron:~/Dropbox/Workspace/Teaching/ENSIA/MobileDev/MyLectures/W13/firebase_message_hello_world$ flutterfire configure
# Fetching available Firebase projects...
i Found 4 Firebase projects.
✓ Select a Firebase project to configure your Flutter application with · <create a new project>
? Enter a project id for your new Firebase project (e.g. my-cool-project) > red-green-screen-notify
```

```

1 Round 4 Firebase projects.
✓ Select a Firebase project to configure your Flutter application with · <create a new project>
✓ Enter a project id for your new Firebase project (e.g. my-cool-project) · red-green-screen-notify
i New Firebase project red-green-screen-notify created successfully.
✓ Which platforms should your configuration support (use arrow keys & space to select)? · android
i Firebase android app com.example.firebase_message_hello_world is not registered on Firebase project red-green-screen-notify.
i Registered a new Firebase android app on Firebase project red-green-screen-notify.
? The files android/build.gradle & android/app/build.gradle will be updated to apply Firebase configuration and gradle build plugins. Do you want to continue? · yes
✓ The files android/build.gradle & android/app/build.gradle will be updated to apply Firebase configuration and gradle build plugins. Do you want to continue? · yes

Firebase configuration file lib/firebase_options.dart generated successfully with the following Firebase apps:

Platform  Firebase App Id
android   1:317332762100:android:6bdd58badc840e92fadda1

Learn more about using this file and next steps from the documentation:
> https://firebase.google.com/docs/flutter/setup
imed@imed-Inspiron:~/Dropbox/Workspace/Teaching/ENSIA/MobileDev/MyLectures/W13/firebase_message_hello_world$

```

- Make sure that you have installed the firebase cli tools
- Within your project terminal run:
 - **flutterfire configure**
 - Either choose an existing project or create a new one
 - You need to choose the platform you want to deploy the app (Android only ?)

01

File Edit Selection View Go Run Terminal Help



EXPLORER

...

main.dart

red.dart 1

firebase_options.dart X

green.dart 1

v FIREBA... [Add] [New Folder] [Refresh] [Copy]

> .dart_tool

> .idea

> android

> build

> ios

v lib

v screens

green.dart 1

red.dart 1

firebase_options.d...

main.dart

> linux

> macos

> web

> windows

≡ .flutter-plugins

≡ .flutter-plugins-depe...

◆ .gitignore

≡ .metadata

! analysis_options.yaml

lib > firebase_options.dart > ...

```
1 // File generated by FlutterFire CLI.
2 // ignore_for_file: lines_longer_than_80_chars, avoid_classes_with_only_stat
3 import 'package:firebase_core/firebase_core.dart' show FirebaseOptions;
4 import 'package:flutter/foundation.dart'
5     show defaultTargetPlatform, kIsWeb, TargetPlatform;
6
7 /// Default [FirebaseOptions] for use with your Firebase apps.
8 ///
9 /// Example:
10 /// ```dart
11 /// import 'firebase_options.dart'
12 /// // ...
13 /// await Firebase.initializeApp(
14 ///   options: DefaultFirebaseOptions.currentPlatform,
15 /// );
16 /// ```
17 class DefaultFirebaseOptions {
18   static FirebaseOptions get currentPlatform {
19     if (kIsWeb) {
20       throw UnsupportedError(
21         'DefaultFirebaseOptions have not been configured for web - '
22         'you can reconfigure this by running the FlutterFire CLI again.',
23       );
24     }
25   }
26 }
```

Firebase_options.dart file will be generated

Firebase Services : Messaging

- **Firebase Messaging : Simple Steps for a Hello World !**
 - **Mobile App :**
 - Install the firebase plugins :
 - **`flutter pub add firebase_core`**
 - **`flutter pub add firebase_messaging`**
 - In addition to another plugin for showing messages:
 - **`flutter pub add flutter_local_notifications`**

02

Firebase Services : Messaging

- **Firebase Messaging : Simple Steps for a Hello World !**
 - **Mobile App :**

03

```
Future<bool> my_init_app() async {
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
  await FirebaseMessaging.instance.getInitialMessage();
  init_firebase_messaging();
  FirebaseMessaging.onBackgroundMessage(_firebaseMessagingBackgroundHandler);
  return true;
}

Future<bool> _firebaseMessagingBackgroundHandler(RemoteMessage message) async {
  print("Running Background Message>>>");
  print("Receieved Background message ${message.messageId} - ${message.toMap().toString()}");
  return true;
}
```

Firebase Services : Messaging

- Firebase Mes

- Mobile App



```
Future<bool> init_firebase_messaging() async {
  await firebase_requestPermission();
  FirebaseMessaging.onMessageOpenedApp.listen(_handleFirebaseMessage);

  await FirebaseMessaging.instance.getToken().then((token) {
    print("Firebase token is : $token");
    //Send it to Backend if you want to use it in the future ?
  });
  return true;
}

Future<bool> _handleFirebaseMessage(RemoteMessage message) async {
  print("\n\n\n####Calling FireMessage Opening Handler..\n\n\n");
  return true;
}
```

03

A Dart VM Service on SM S918B is available at: http://127.0.0.1:39303/oEsZTZ-xzLI=

The Flutter DevTools debugger and profiler on SM S918B is available at: http://127.0.0.1:9102?uri=http://127.0.0.1:39303/oEsZTZ-xzLI=

I/flutter (32096): Firebase token is : cCYINRaVQ6ypavdfobnphZ:APA91bG12CLeeps-0g9dkp5LudDFNblWv4TTYFBWcJJZNBMdKi-cnq2Fm7Zp2v31S2b8I_ZUj5um44954dLL5yM9hQeDq2w6dqCHbJJ_zgDPfyHozdksS7cWBylKDiaV7SpupqB1Yb1xE

I/viewRootImp[003/2054[MainActivity]](32096): handleAppVisibility mAppVisible = true visible = false

T/SurfaceView@d234000(32096): onWindowVisibilityChanged(8) false io.flutter.embedding.android.FlutterSurfaceView@d234000 V/E

A A 1000 ?

Firebase Services : Messaging

- **Firebase Messaging : Simple Steps for a Hello World !**

- **Mobile App :**

- Initialize the Firebase Instance : **Getting errors ?**

```
imed@imed-Inspiron:~/Dropbox/Workspace/Teaching/ENSIA/MobileDev/MyLectures/W13/firebase_message_hello_world$ flutter run
Launching lib/main.dart on SM S918B in debug mode...

FAILURE: Build failed with an exception.

* What went wrong:
Execution failed for task ':app:mapDebugSourceSetPaths'.
> Error while evaluating property 'extraGeneratedResDir' of task ':app:mapDebugSourceSetPaths'
   > Failed to calculate the value of task ':app:mapDebugSourceSetPaths' property 'extraGeneratedResDir'.
      > Querying the mapped value of provider(java.util.Set) before task ':app:processDebugGoogleServices' has completed is not supported

* Try:
> Run with --stacktrace option to get the stack trace.
> Run with --info or --debug option to get more log output.
> Run with --scan to get full insights.

* Get more help at https://help.gradle.org

BUILD FAILED in 1s
```

03

Fireb M

android/app/
build.gradle.kts

03

```
plugins {  
    id("com.android.application")  
    // START: FlutterFire Configuration  
    id("com.google.gms.google-services")  
    // END: FlutterFire Configuration  
    id("kotlin-android")  
    // The Flutter Gradle Plugin must be applied after the Android and Kotlin Gradle plugins.  
    id("dev.flutter.flutter-gradle-plugin")  
}  
  
android {  
    namespace = "com.example.flutter_hello_app"  
    compileSdk = 35  
    ndkVersion = flutter.ndkVersion  
  
    compileOptions {  
        isCoreLibraryDesugaringEnabled = true  
        sourceCompatibility = JavaVersion.VERSION_17  
        targetCompatibility = JavaVersion.VERSION_17  
    }  
}  
  
flutter {  
    source = "../.."  
}  
  
dependencies {  
    // PUT IT HERE ← in android/app/build.gradle.kts  
    coreLibraryDesugaring("com.android.tools:desugar_jdk_libs:2.1.4")  
}  
  
// Apply Google Services plugin (add at the end)  
apply(plugin = "com.google.gms.google-services")
```

Firek

M

android/build
d.gradle.kts

03

```
buildscript {
    repositories {
        google()
        mavenCentral()
    }

    dependencies {
        classpath("com.android.tools.build:gradle:8.1.0")
        classpath("org.jetbrains.kotlin:kotlin-gradle-plugin:1.9.0")
        classpath("com.google.gms:google-services:4.4.0")
    }
}

allprojects {
    repositories {
        google()
        mavenCentral()
    }
}

val newBuildDir: Directory =
    rootProject.layout.buildDirectory
        .dir("../..../build")
        .get()
rootProject.layout.buildDirectory.value(newBuildDir)

subprojects {
    val newSubprojectBuildDir: Directory = newBuildDir.dir(project.name)
    project.layout.buildDirectory.value(newSubprojectBuildDir)
}

subprojects {
    project.evaluationDependsOn(":app")
}
```

Firebase Services : Messaging

- **Firebase Messaging : Simple Steps for a Hello World !**
 - **Mobile App :**
 - Testing using Firebase Cloud Messaging Console :
 - Using the obtained token
 - We send a hello message

03

```
A Dart VM Service on SM S918B is available at: http://127.0.0.1:39303/oEsZTZ-xzLI=/
The Flutter DevTools debugger and profiler on SM S918B is available at: http://127.0.0.1:9102?uri=http://127.0.0.1:39303/oEsZTZ-xzLI=/
I/flutter (32096): Firebase token is : cCYINRaVQ6ypavdfobnphZ:APA91bG12CLeePs-0g9dkp5LudDFNblWv4TTYFBWcJJZNBmDkI-cnq2Fm7Zp2v31S2b8I_ZUj5um44954dLL5yM
9hQeDq2w6dqCHbJJ_zgDPfyHozdksS7cWBylKDiaV7SpupqB1Yb1xE
I/viewRootImp@03/2054[MainActivity](32096): handleAppVisibility mAppVisible = true visible = false
T/SurfaceView@d234000(32096): onWindowVisibilityChanged(8) false io.flutter.embedding.android.FlutterSurfaceView@d234000 V E
```



Project shortcuts


Messaging

Product categories

Build
Release & Monitor
Analytics
Engage

All products

red-green-screen-notify

Cloud Messaging

Compose notification

1 Notification

Notification title

Notification text

Notification image (optional)



Notification name (optional)

Device preview

This preview provides a general idea of how the notification will appear on a mobile device. Actual message may vary based on the device. Test with a real device for more details.

Send test message
Initial state

Expanded view

TEST Title

This is a test message sent from the Console

And

4dLL5yM

1080 2

Fireba Me

- **Firestore Mess:**
 - **Mobile App**

4 Additional options (optional)

All fields optional

Android Notification Channel ?

Custom data ?

screen

green

Key

Value

- Testing using Firebase Cloud Messaging Console :
 - Beside the title and body, Firebase Message may (**ALWAYS**) have an extra data payload in the form a dictionary/map

03



Project shortcuts


 Messaging

Product categories

Build 

Release & Monitor 

red-green-screen-notify ▾

Cloud Messaging

Compose notification

1 Notification

Notification title 

TEST Title

text

test message sent from the Console

general idea

l message

l device for

image (optional) 

https://yourapp.com/image.png



name (optional) 

optional name

Device preview

This preview provides a general idea of how the message will appear on a mobile device. Actual message may vary.

Send test message

Initial state

Expanded view

TEST Title

This is a test message sent from the Console

4dLL5yM

1080 2

Test on device

You can test this campaign by entering or selecting the [FCM registration tokens](#)  of your development device below.

.L5yM9hQeDq2w6dqCHbJJ_zgDPfyHozdksS7cWBylKDiaV7SpupqB1Yb1xE 

No test devices configured

Cancel

Test

```

Future<bool> my init app() async {
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
  await FirebaseMessaging.instance.getInitialMessage();
  init_firebase_messaging();
  FirebaseMessaging.onBackgroundMessage(_firebaseMessagingBackgroundHandler);
  return true;
}

Future<bool> firebaseMessagingBackgroundHandler(RemoteMessage message) async {
  print("Running Background Message>>>");

  print("Receieved Background message ${message.messageId} - ${message.toMap().toString()}");

  return true;
}

```

I/flutter (24901): Running Background Message>>>

W/FirebaseMessaging(24901): Unable to log event: analytics library is missing

I/flutter (24901): Receieved Background message 0:1702999589768848%8028adc68028adc6 - {senderId: null, category: null, collapseKey: com.example.firebse_message_hello_world, contentAvailable: false, data: {screen: red}, from: 317332762100, messageId: 0:1702999589768848%8028adc68028adc6, messageType: null, mutableContent: false, notification: {title: TEST Title, titleLocArgs: [], titleLocKey: null, body: This is a test message sent from the Console, bodyLocArgs: [], bodyLocKey: null, android: {channelId: null, clickAction: null, color: null, count: null, imageUrl: null, link: null, priority: 0, smallIcon: null, sound: null, ticker: null, tag: campaign_collapse_key_8496497378862318570, visibility: 0}, apple: null, web: null}, sentTime: 170299589759, threadId: null, ttl: 2419200}

W/FirebaseMessaging(24901): Missing Default Notification Channel metadata in AndroidManifest. Default value will be used.

Development device below.

.L5yM9hQeDq2w6dqCHbJJ_zgDPfyHozdksS7cWBBylKDiaV7SpupQB1Yb1xE

No test devices configured

Cancel

Test

tion name (optional) ?

optional name

TEST Title

This is a test message sent from t

4dLL5yM

108A 2

```

Future<bool> my init app() async {
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
  await FirebaseMessaging.instance.getInitialMessage();
  init_firebase_messaging();
  FirebaseMessaging.onBackgroundMessage(_firebaseMessagingBackgroundHandler);
  return true;
}

Future<bool> firebaseMessagingBackgroundHandler(RemoteMessage message) async {
  print("Running Background Message>>>");

  print("Receieved Background message ${message.messageId} - ${message.toMap().toString()}");

  return true;
}

```

I/flutter (24901): Running Background Message>>>

W/FirebaseMessaging(24901): Unable to log event: analytics library is missing

I/flutter (24901): Receieved Background message 0:1702999589768848%8028adc68028adc6 - {senderId: null, category: null, collapseKey: com.example.firebse_message_hello_world, contentAvailable: false, data: {screen: red}, from: 317332762100, messageId: 0:1702999589768848%8028adc68028adc6, messageType: null, mutableContent: false, notification: {title: TEST Title, titleLocArgs: [], titleLocKey: null, body: This is a test message sent from the Console, bodyLocArgs: [], bodyLocKey: null, android: {channelId: null, clickAction: null, color: null, count: null, imageUrl: null, link: null, priority: 0, smallIcon: null, sound: null, ticker: null, tag: campaign_collapse_key_8496497378862318570, visibility: 0}, apple: null, web: null}, sentTime: 170299589759, threadId: null, ttl: 2419200}

W/FirebaseMessaging(24901): Missing Default Notification Channel metadata in AndroidManifest. Default value will be used.

Development device below.

.L5yM9hQeDq2w6dqCHbJJ_zgDPfyHozdksS7cWBylKDiaV7SpupqB1Yb1xE

No test devices configured

Cancel

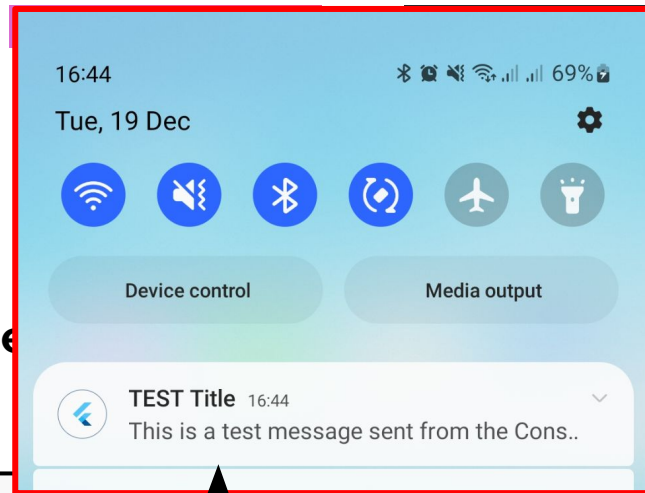
Test

The app needs to be in the **background** or **closed**

Firebase Services : Messaging

- **Firebase Messaging : Simple Steps for a Hello World**

- **Mobile App :**



```
I/flutter (24901): Running Background Message>>>
```

```
W/FirebaseMessaging(24901): Unable to log event: analytics library is missing
```

```
I/flutter (24901): Received Background message 0:1702999589768848%8028adc68028adc6 - {senderId: null, category: null, collapseKey: com.example.fireb  
se_message_hello_world, contentAvailable: false, data: {screen: red}, from: 317332762100, messageId: 0:1702999589768848%8028adc68028adc6, messageType  
null, mutableContent: false, notification: {title: TEST Title, titleLocArgs: [], titleLocKey: null, body: This is a test message sent from the Conso  
e, bodyLocArgs: [], bodyLocKey: null, android: {channelId: null, clickAction: null, color: null, count: null, imageUrl: null, link: null, priority: 0  
smallIcon: null, sound: null, ticker: null, tag: campaign_collapse_key_8496497378862318570, visibility: 0}, apple: null, web: null}, sentTime: 17029  
9589759, threadId: null, ttl: 2419200}
```

```
W/FirebaseMessaging(24901): Missing Default Notification Channel metadata in AndroidManifest. Default value will be used.
```

Development device below.

.L5yM9hQeDq2w6dqCHbJJ_zgDPfyHozdksS7cWBylKDiaV7SpupqB1Yb1xE

No test devices configured

Cancel

Test

What happens when they click on the notification message ?

= true visible = false

T/SurfaceView@d234A9A(234A9A): onWindowVisibilityChanged(8) false in flutter.embedding.android.FlutterSurfaceView@d234A9A V E

A A 1A8A 2

Firebase Services : Messaging

- **Firebase Messaging : Simple Steps for a Hello World !**

- **Mobile App :**

- Custom Navigation from Background Notification:

- If inside the data payload,

- the screen value is red $\Rightarrow$ show red screen
- the screen value is green $\Rightarrow$ show green screen
- Otherwise, just do the default behaviour

04

utils/firebase.dart

```
Future<bool> handleFirebaseMessage(RemoteMessage message) async {
  print("\n\n\n#####Calling FireMessage Opening Handler...\n\n\n");
  Map<String, dynamic> data = Map.of(message.data);

  if (data.containsKey('screen') && data['screen'] == 'red') {
    Navigator.of(navigatorKey.currentContext!).push(MaterialPageRoute(
      builder: (context) => RedScreen(
        message: message.notification!.body.toString(),
      )),);
  } else if (data.containsKey('screen') && data['screen'] == 'green') {
    Navigator.of(navigatorKey.currentContext!).push(MaterialPageRoute(
      builder: (context) => GreenScreen(
        message: message.notification!.body.toString(),
      )),);
  }
  return true;
}
```

main.dart

```
final GlobalKey<NavigatorState> navigatorKey = GlobalKey<NavigatorState>();
```

```
@override
Widget build(BuildContext context) {
  return MaterialApp(
    home: RedScreen(message: 'Initial Loading..as RED'),
    navigatorKey: navigatorKey,
  );
}
```


utils/firebase.dart

```
Future<bool> handleFirebaseMessage(RemoteMessage message) async {  
  print("\n\n\n#####Calling FireMessage Opening Handler...\n\n\n");  
  Map<String, dynamic> data = Map.of(message.data);  
  
  if (data.containsKey('screen') && data['screen'] == 'red') {  
    Navigator.of(navigatorKey.currentContext!).push(MaterialPageRoute(  
      builder: (context) => RedScreen(  
        message: message.notification!.body.toString(),  
      )),  
    );  
  } else if (data.containsKey('screen') && data['screen'] == 'green') {  
    Navigator.of(navigatorKey.currentContext!).push(MaterialPageRoute(  
      builder: (context) => GreenScreen(  
        message: message.notification!.body.toString(),  
      )),  
    );  
  }  
  return true;  
}
```

main.dart

**For handling notifications when clicked
only when the App is CLOSED or not in
foreground..**

```
final GlobalKey<NavigatorState> navigatorKey = GlobalKey<NavigatorState>();
```

```
@override  
Widget build(BuildContext context) {  
  return MaterialApp(  
    home: RedScreen(message: 'Initial Loading..as RED'),  
    navigatorKey: navigatorKey,  
  );  
}
```

Firebase Services : Messaging

- **Firebase Messaging : Simple Steps for a Hello World !**

- **Mobile App :**

- **Receiving In-App Messages :**

- Initialize the Local Notification Plugin Instance
- Need to listen onMessage from Firebase
- Integrate Local Notification Plugin to show message
- Handle Navigation if needed.

05

Firebase Services : Messaging

main.dart

```
FlutterLocalNotificationsPlugin flutterLocalNotificationsPlugin =  
    FlutterLocalNotificationsPlugin();
```

- **Firebase Messaging : Simple Steps for a Hello World !**

utils/firebase.dart

- ```
Future<bool> init_firebase_messaging() async {
 await firebase requestPermission();
 await firebase_initialize_local_plugin();
 . . .
}
```

- Integrate Local Notification Plugin to show message
- Handle Navigation if needed.

05

```

firebase_initialize_local_plugin() {
 var androidInitialize =
 const AndroidInitializationSettings('@mipmap/ic_launcher');
 var initializationSettings =
 InitializationSettings(android: androidInitialize);
 flutterLocalNotificationsPlugin.initialize(initializationSettings,
 onDidReceiveNotificationResponse:
 (NotificationResponse notificationResponse) {
 try {
 if (notificationResponse.payload != null &&
 notificationResponse.payload!.isNotEmpty) {
 Map<String, dynamic> payload_data =
 jsonDecode(notificationResponse.payload!);
 Map<String, dynamic> data = payload_data['data'];
 if (data!['screen'] == 'red') {
 Navigator.of(navigatorKey.currentContext!).push(MaterialPageRoute(
 builder: (context) => RedScreen(
 message: 'From from an In-app message',
)),);
 }
 if (data!['screen'] == 'green') {
 Navigator.of(navigatorKey.currentContext!).push(MaterialPageRoute(
 builder: (context) => GreenScreen(
 message: 'From from an In-app message',
)),);
 }
 }
 }
 }
 on Exception catch (e, stack) {
 print('Exception $e $stack');
 }
 });
}

```

utils/firebase.dart

```

FirebaseMessaging.onMessage.listen((RemoteMessage message) async {
 BigTextStyleInformation bigTextStyleInformation = BigTextStyleInformation(
 message.notification!.body.toString(),
 htmlFormatBigText: true,
 contentType: message.notification!.title.toString(),
 htmlFormatTitle: true,
);
 AndroidNotificationDetails AndroidplatformChannelSpecifics =
 AndroidNotificationDetails(
 'Red Green Screen App',
 'ENSIA APP',
 importance: Importance.high,
 styleInformation: bigTextStyleInformation,
 priority: Priority.high,
 playSound: true,
);

 NotificationDetails platformChannelSpecifics =
 NotificationDetails(android: AndroidplatformChannelSpecifics);

 try {
 await flutterLocalNotificationsPlugin.show(0, message.notification!.title,
 message.notification!.body.toString(), platformChannelSpecifics,
 payload: jsonEncode(message.toMap()));
 } on Exception catch (e, stack) {
 print('Exception $e $stack');
 }
});

```

utils/firebase.dart

# Firebase Services : Messaging

- **Firebase Messaging : Simple Steps for a Hello World !**

- **Mobile App :**

- Subscribing users to **Topics** :

- When the user visits The red screen, we subscribe them to the **Red** Topic and the same for **Green**.

- Instructions of Firebase Code :

- `await FirebaseMessaging.instance.subscribeToTopic("Red");`
- `await FirebaseMessaging.instance.unsubscribeFromTopic("Green");`

06

# Firebase Services : Messaging

Revision : Where to inject those  
two lines of code ?

- **Firebase Messaging : Simple Steps for a Hello World !**

- **Mobile App :**

- Subscribing users to **Topics** :

- When the user visits The red screen, we subscribe them to the **Red** Topic and the same for **Green**.

- Instructions of Firebase Code :

- `await FirebaseMessaging.instance.subscribeToTopic("Red");`
- `await FirebaseMessaging.instance.unsubscribeFromTopic("Green");`

06

# Firebase Services :

```
class _RedScreenState extends State<RedScreen> {

 @override
 void initState() {
 super.initState();
 subscribeUserToTopic("Red");
 }
}
```

or a Hello World !

## ○ Mobile App



```
Future<bool> subscribeUserToTopic(String topic) async {
 print("Subscribing to Topic $topic");
 try {
 if (topic == "Red") {
 await FirebaseMessaging.instance.subscribeToTopic("Red");
 await FirebaseMessaging.instance.unsubscribeFromTopic("Green");
 }
 if (topic == "Green") {
 await FirebaseMessaging.instance.subscribeToTopic("Green");
 await FirebaseMessaging.instance.unsubscribeFromTopic("Red");
 }
 } on Exception catch (e, stack) {
 print('$e $stack');
 }
 return true;
}
```

06

# Firebase Services :

```
class _RedScreenState extends State<RedScreen> {

 @override
 void initState() {
 super.initState();
 subscribeUserToTopic("Red");
 }
}
```

How to test sending to specific topic ?

or a Hello World !

## ○ Mobile App



```
Future<bool> subscribeUserToTopic(String topic) async {
 print("Subscribing to Topic $topic");
 try {
 if (topic == "Red") {
 await FirebaseMessaging.instance.subscribeToTopic("Red");
 await FirebaseMessaging.instance.unsubscribeFromTopic("Green");
 }
 if (topic == "Green") {
 await FirebaseMessaging.instance.subscribeToTopic("Green");
 await FirebaseMessaging.instance.unsubscribeFromTopic("Red");
 }
 } on Exception catch (e, stack) {
 print('$e $stack');
 }
 return true;
}
```

06

# Firestore Services : Messaging

- **Firestore Messaging : Simple Steps for a Hello World !**
  - **Testing the Message :**
    - If you are using Firestore Console, you need to be patient as it takes time.
    - Prefer to use Python Client



# Firebase Services : Messaging

- **Firebase Messaging : Simple Steps for a Hello World !**

- **Testing the Message :**

- Steps for testing using a Hello World Python Client

- Install the FirebaseSDK for your preferred language (Java, Python ..)

- Python : **sudo pip3 install firebase\_admin**

- Download the Credentials JSON file from your project
      - Write the Hello World

# Project settings

General Cloud Messaging Integrations **Service accounts** Data privacy Users and permissions

[Manage service account permissions](#)

## Firebase Admin SDK

Legacy credentials

## Database secrets

All service accounts

## [3 service accounts](#)

### Firebase Admin SDK

Your Firebase service account can be used to authenticate multiple Firebase features, such as Database, Storage and Auth, programmatically via the unified Admin SDK. [Learn more](#)

Firebase service account  
firebase-adminsdk-4oab4@red-green-screen-notify.iam.gserviceaccount.com

Admin SDK configuration snippet

☐ Node.js ☐ Java ☒ Python ☐ Go

```
import firebase_admin
from firebase_admin import credentials

cred = credentials.Certificate("path/to/serviceAccountKey.json")
firebase_admin.initialize_app(cred)
```

[Generate new private key](#)

# Firebase Services :

## Messaging

```
from firebase_admin import messaging, credentials
import firebase_admin

creds = credentials.Certificate('Imed_Private_JSON_FILE.json')

default_app = firebase_admin.initialize_app(creds)

topic='Red'
title = 'Hello Red/Green'
body = 'From to Red/Green App from Python'
data = {
 'screen': 'green'
}
message = messaging.Message(
 notification=messaging.Notification(
 title=title,
 body=body,
),
 data=data,
 topic=topic
)
response = messaging.send(message)
print(str(response))
```

# Firebase Services :

## Messaging

- **Firebase Messaging : How to Integrate with my Project**

- **Case Study :**

- **ENSIA Social App :**

- When someone publishes a post to a specific category,
        - All users following that app, will receive a notification about this post
        - When, they click on the notification box, it will take them to another screen.

# Firestore Services : Messaging

- **Firestore Messaging : How to Integrate with my Project**

- **Case Study :**

- **ENSIA Social App :**

MobileApp  
User publishes  
Post

notify.subscribers

Python Backend

If you are Certain that the execution will be quick : Send to Firestore Servers directly.

Otherwise, queue approach, store into a database/queue and process them incrementally

# Firebase Services : Messaging

- **Firebase Messaging : How to Integrate with my Project**
  - **Case Study :**
    - **All Projects :**
      - You release a new version of the App, you want to send kind message for all users to download to upgrade to the latest version ?
      - You have some news to share with your users ?

# Firebase Services : Messaging

- **Firebase Messaging : How to Integrate with my Project**

- **Case Study :**

- **All Projects :**

- You release a new version of the App, you want to send kind message for all users to download to upgrade to the latest version ?
      - You have some news to share with your users ?
      - Users **who didn't use the app for a month or more**, send them a promotional message to get them back to use your app

# Firestore Services : Messaging

- **Firestore Messaging : How to learn ?**
  - Here is the MVP without Messaging Integration
    - Link to Dropbox here ?
  - **Add the Firestore Cloud Messaging to it on your own**
  - Final App with messaging Integrated:
    - Link to Dropbox here..





# External Background Services

# Section 3

## Permissions & Accessing the Hardware

# Permissions & Accessing the Hardware



- **Android & Permissions :**

- Android protects sensitive hardware resources including:
  - *Camera*
  - *Location*
  - *Microphone*
  - *Storage*
  - *Phone state*
  - *Other sensors...*
- Permission must be obtained from the user to access such resources

# Permissions & Accessing the Hardware

- **Android & Permissions :**

- For older versions of Android, the permission must be explicitly defined inside the file **android/app/src/main/AndroidManifest.xml**

```
<uses-permission android:name="android.permission.CAMERA"/>
```

```
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
```

```
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
```

# Permissions & Accessing the Hardware



- **Android & Permissions :**
  - Newer versions are asking for permissions on the go when they are needed for **first time**.

# Permissions & Accessing the Hardware



- **Android & Permissions :**

- If the user defined permission for a certain hardware, You need to manually ask for their permission if this access is needed for the functionalities
- The **permission\_handler** package can be used.

```
Future<void> requestCameraPermission() async {
 var status = await Permission.camera.status;

 if (!status.isGranted) {
 status = await Permission.camera.request();
 }

 if (status.isGranted) {
 print("Camera permission granted");
 } else if (status.isDenied) {
 print("Camera permission denied");
 } else if (status.isPermanentlyDenied) {
 openAppSettings();
 }
}
```



```
Future<void> pickImage() async {
 var status = await Permission.photos.status; // For gallery

 if (status.isDenied) {
 status = await Permission.photos.request();
 }

 if (status.isPermanentlyDenied) {
 openAppSettings(); // Redirect to app settings
 return;
 }

 if (status.isGranted) {
 final picker = ImagePicker();
 final picked = await picker.pickImage(source: ImageSource.gallery);

 if (picked != null) {
 print("Image selected: ${picked.path}");
 } else {
 print("User canceled picking.");
 }
 }
}
```

